Why Only One Hand Worked:
Bimanual Cross-Talk, Latency and Snap in MABEL's Vision Pro Whole-Body Retargeting
(`mabel.teleop_engine`, `mabel.whole_body_retargeting`)

MABEL Project — Control & Teleoperation

2026-07-31

Abstract

An operator reported four symptoms of MABEL's Vision Pro whole-body teleoperation at once: severe delay, only one hand ever driving the robot, targets being reached too abruptly, and intermittent oscillation — with self-collision avoidance suspected as the common cause. We isolate each claim with a synthetic ARKit operator that is byte-compatible with the headset's wire format, so no headset, network or physical robot enters the measurement. Three of the four claims resolve to causes unrelated to the suspicion, and the suspicion itself is falsified. The one-hand symptom has *two* independent causes: a client-side hand pose that never expires, and a body-frame estimator that reads a two-handed lateral sweep as a torso turn. The latter injected 20.2° of phantom torso yaw under a rigidly static head, and because arm targets are decomposed in that frame it rotated *both* arms — each arm then tracked the *other* hand ($r = -0.84$) better than its own ($r = +0.36$). A conjunction gate (no wrist-orientation evidence *and* a still head) removes it: phantom yaw 20.2°→0.00°, own-hand tracking $+0.36 \rightarrow +0.88$, cross-talk $-0.84 \rightarrow -0.15$. Self-collision avoidance is shown to be a control-barrier boundary layer, not a potential field: it settles with exactly zero residual command velocity, and its contribution to steady-state command jitter is identically zero to three decimal places at every noise level tested. The retargeting engine itself adds ≤ 12 ms of phase lag at every client frame rate from 15 Hz to 90 Hz, so it is not the source of the reported delay; the recoverable client-side latency is identified instead.

Keywords: teleoperation, whole-body retargeting, bimanual manipulation, body-frame estimation, control barrier functions, self-collision avoidance, latency.

1 Overview

MABEL is driven from an Apple Vision Pro: ARKit head and two 27-joint hand skeletons stream to a server, where `TeleopEngine` maps them onto a 7-DoF arm pair, hands, neck and mobile base. On 2026-07-31 the operator reported, in one sentence, four distinct failures — delay, single-hand operation, an over-abrupt approach to target, and oscillation — and hypothesised that the self-collision avoidance was responsible.

Four claims sharing one hypothesis is exactly the situation in which measuring the hypothesis first is wasteful: it is one candidate out of many, and three of the claims turn out not to involve it. We instead built a synthetic operator and tested each claim independently.

1.1 Method: a synthetic ARKit operator

`SyntheticVPOperator` (`controller/experiments/exp_vp_bimanual_latency.py`) emits `OperatorPose` frames in the ARKit convention (x right, y up, $-z$ forward) carrying, per side, a full named hand skeleton and the `forearm_arm` elbow the wire carries. This is the same structure `RobotController.makeFrameSource` places on the wire, so the engine cannot distinguish it from a headset. Anthropometry (shoulder width, upper-arm and forearm lengths) is explicit; per-side wrist trajectories are prescribed; ARKit jitter is a seeded Gaussian. Every number below is therefore reproducible and attributable: what the harness reproduces is owned by the engine, and what it does not is owned by the client or the transport.

The engine runs against an ideal plant ($q := u$) so that servo dynamics do not contaminate retargeting measurements.

1.2 Two measurement pitfalls worth recording

Both cost real time and both would have produced confident wrong answers.

Comparing like-named axes across frames. The operator signal lives in the ARKit frame and the robot signal in the robot world, related by the retargeting map — a similarity transform with cross-axis mixing. Correlating operator-x against robot-x made a perfectly tracked left arm appear *anti*-correlated, and a cross-correlation lag scan then locked onto whatever shift restored a positive sign, reporting a physically impossible 190 ms phase *lead*. Projecting each trajectory onto its own first principal component removes the frame dependence entirely; lag is then recovered by single-frequency (lock-in) demodulation at the frequency we generated.

Differentiating at the wrong rate. The command steps once per operator frame (33 ms) while the record is sampled every control tick (5 ms), so a per-frame trust step of 0.13 rad read as $26.7\,\mathrm{rad\,s^{-1}}$ — an artefact of sampling, not a speed the arm ever holds. Velocities are differentiated at the frame rate.

2 Bimanual cross-talk (the one-hand symptom)

2.1 Cause 1 (client): a hand pose that never expires

ARKit delivers one chirality per anchor update, so `HandTrackingService.startBroadcast` carries `left`/`right` across loop iterations in order to emit both hands on every snapshot. That merge is correct and load-bearing. But the two locals were only ever *assigned*. ARKit may simply stop emitting updates for a hand that leaves the field of view rather than emitting one with `isTracked == false`, in which case that hand kept streaming its last good pose — `isTracked: true`, full skeleton — at the full frame rate indefinitely.

The server drops a hand only when `isTracked` is false (`mabel/sources/wire.py`, `_hand_world`), so it continued retargeting that arm to a frozen target while the other hand moved normally. The stuck arm is whichever hand left view first, and it never recovers.

The fix is per-side refresh stamps expired at both ends: in the producer, so a mixed one-hand-gone case clears immediately, and at read time on the 60 Hz wire tick, because when *both* hands leave no anchor update arrives at all and the producer never runs again. Pinned by `MabelTeleopTests/HandStalenessTests`.

2.2 Cause 2 (engine): a two-arm sweep read as a torso turn

The engine decomposes arm targets in the operator's *body* frame (`ARM_BODY_FRAME_MAP`) so that the same body-relative reach produces the same robot configuration regardless of where the operator is standing or facing. The body frame's yaw is estimated by `BodyFrameEstimator` from the head and *both* wrists. Coupling is therefore structural: if one hand's motion moves the estimate, it rotates the other arm's target.

Bisecting the candidate stages (`probe_arm_crosstalk.py`) isolates it unambiguously. Table 1 toggles one stage at a time; only the body-frame map matters, and disabling self-collision or the bimanual proximity blend changes nothing at the third decimal.

Table 1: Bimanual cross-talk bisection. Both hands sweep at their own frequency (0.55 Hz right, 0.35 Hz left); the two operator signals are themselves uncorrelated ($r = -0.09$), so cross-correlation is not an artefact. "own" is each arm against its own hand, "cross" against the other hand.

configuration	own r (right)	cross r (right $\leftarrow$ left)
baseline (before fix)	$+0.363$	-0.835
body-frame map OFF	$+0.824$	-0.341
bimanual blend + floor OFF	$+0.361$	-0.835
self-collision OFF	$+0.358$	-0.834
all OFF	$+0.825$	-0.341
baseline (after fix)	$\mathbf{+0.878}$	$\mathbf{-0.149}$

2.3 Why every existing guard missed it

`BodyFrameEstimator` fuses two cues: the common-mode change of the two wrist *bearings* about the chest (θ_{pos}) and the common-mode incremental yaw of the two wrist *orientations* (θ_{wori}). Its parallel-sweep detector is

$$\text{anti} = \text{clip}\left(\frac{-2\,\theta_{\text{pos}}\,\theta_{\text{wori}}}{\theta_{\text{pos}}^2 + \theta_{\text{wori}}^2}, 0, 1\right), \tag{1}$$

which is *defined* from θ_{wori}. When the hands translate without rotating — both hands sweeping sideways, carrying a tray, pushing with two flat palms — we have $\theta_{\text{wori}} \approx 0$, so anti $\to 0$, the head-corroboration branch it gates is never taken, and the bearing cue alone is trusted ungated. Two wrists swinging the same way about the chest is precisely what a body turn looks like to that cue.

Measured with a rigidly static head and torso, only the hands moving (`probe_body_yaw_swing.py`), every degree of which is error:

Table 2: Phantom torso yaw under a rigidly static head and torso.

motion	before	after
both hands sweeping	$20.22°$ (std $4.63°$)	$0.00°$
right hand only	$0.00°$	$0.00°$
left hand only	$0.00°$	$0.00°$
hands still (control)	$0.00°$	$0.00°$

The single-hand rows are exactly zero, which is why the defect only ever appeared bimanually — and why it presented to the operator as "one hand works".

2.4 The fix: gate the conjunction, not the evidence

Let $w_{\text{pos}}, w_{\text{wori}}$ be the two cues' confidence weights. Define the orientation-evidence fraction and a head-stillness measure

$$\phi = \frac{w_{\text{wori}}}{w_{\text{wori}} + w_{\text{pos}}}, \qquad \text{blind} = \text{clip}\left(1 - \frac{\phi}{\phi_{\min}}, 0, 1\right), \tag{2}$$

$$\text{still} = \text{clip}\left(1 - \frac{|\theta_{\text{head}}|/\Delta t}{\dot{\psi}_{\text{corr}}}, 0, 1\right), \qquad \theta_{\text{rigid}} \leftarrow \theta_{\text{rigid}}\,(1 - \text{blind} \cdot \text{still}), \tag{3}$$

with $\phi_{\min} = 0.15$ and $\dot{\psi}_{\text{corr}} = 0.15\,\text{rad}\,\text{s}^{-1}$.

Only the *conjunction* is suppressed. Gating on low orientation evidence alone was tried first and rejected by measurement: it also fired on 41 % of the frames of a genuine walk-turn-reach, where a reach transiently starves the orientation cue mid-turn, degrading that case from 0.6° to 3.6° RMS. Head stillness is the discriminator that separates the two situations: during any real turn the head goes with the body, still collapses to zero, and the term vanishes identically. This also keeps the module's stated design rule — the head is a yes/no gate, never a magnitude, so a neck glance can never set the base rate.

All 14 pre-existing body-frame scenarios still pass, plus two new regression tests: an orientation-blind two-arm sweep must leak under 2°, and the same sweep with a corroborating head turn must still turn the body.

3 Latency

3.1 The engine is not the source

Fig. 1 sweeps the client frame rate. Phase lag between the operator's wrist and the commanded robot wrist, by lock-in demodulation on principal-axis projections, is ≤ 12 ms on both sides at every rate from 15 Hz to 90 Hz — below one frame interval, i.e. at the resolution floor of the measurement. Per-frame solve cost is 9 ms to 14 ms median.

The retargeter therefore does not own the reported delay. This independently confirms the 2026-07-24 finding that the arm *servo* gain, not the retargeter, dominated a previous latency complaint.

3.2 Recoverable client-side latency

Auditing the outbound chain identified one clearly recoverable term. The head was sampled at $t + 30$ ms (visionOS extrapolates anchors, so asking ahead cancels transport latency rather than adding to it) while the hands took the last broadcast pose, itself up to 11 ms old at the 90 Hz anchor rate. The hands therefore ran 35 ms to 41 ms behind the head, on the channel the operator watches most, and the neck consequently led the arms through every motion. `HandTrackingService.predictedPair(at:)` had been written for exactly this and was never called; it is now called on the wire queue (never the main actor), gated on the freshness box so a predicted pose cannot resurrect a hand that ARKit has stopped tracking.

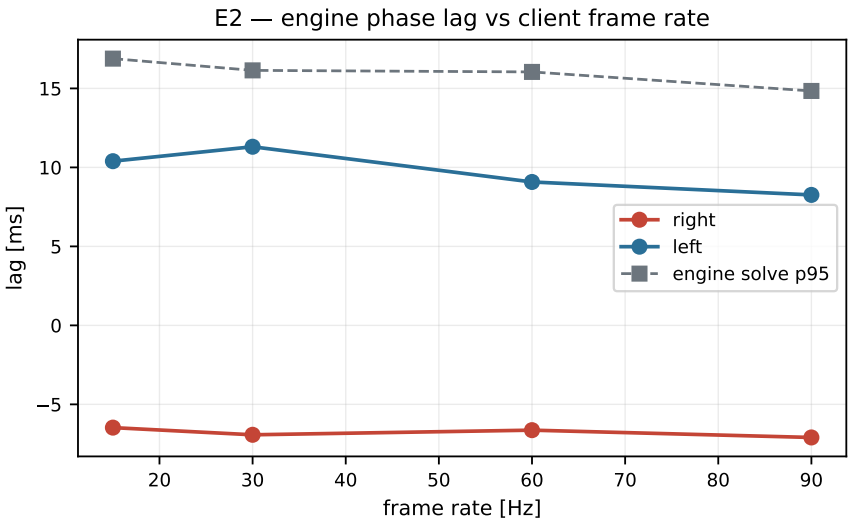


Figure 1: Engine phase lag versus client frame rate, per side, with the per-frame solve cost. Lag stays within one frame interval throughout.

4 Snap

Fig. 2 steps the wrist target by 20 cm. The arm's peak commanded joint speed and the step overshoot both fall with `ARM_TRUST_RATE`, while tracking lag on a continuous sweep does not move at all (Table 3) — because normal tracking never reaches the limit; only a *jump* does (an engage, a re-anchor, a hand returning to view). The snap is therefore bought down for free, which is what allows this to be lowered without trading against the operator's stated first priority.

Table 3: Snap versus lag. Above $\sim 4\,\mathrm{rad\,s^{-1}}$ the trust region is not binding at all (6.0 and 4.0 give an identical peak): the old default was inactive except on the largest jumps, where it was insufficient.

`ARM_TRUST_RATE`	peak joint speed	step overshoot	sweep lag
$6.0\,\mathrm{rad\,s^{-1}}$ (old)	$3.81\,\mathrm{rad\,s^{-1}}$	39.3 %	−7.3 ms
$4.0\,\mathrm{rad\,s^{-1}}$	$3.81\,\mathrm{rad\,s^{-1}}$	41.7 %	−7.4 ms
$2.5\,\mathrm{rad\,s^{-1}}$	$3.24\,\mathrm{rad\,s^{-1}}$	33.2 %	−6.7 ms
$2.0\,\mathrm{rad\,s^{-1}}$ **(new)**	$2.73\,\mathrm{rad\,s^{-1}}$	27.3 %	−6.5 ms
$1.5\,\mathrm{rad\,s^{-1}}$	$2.29\,\mathrm{rad\,s^{-1}}$	21.0 %	−5.1 ms

5 Oscillation, and the self-collision hypothesis

5.1 Self-collision avoidance is a boundary layer, not a potential field

The retargeting path uses `ArmCollisionProjector`, a one-step velocity QP

$$\min_{\dot{q}} \tfrac{1}{2}\|\dot{q} - \dot{q}_{\mathrm{des}}\|^2 \quad \text{s.t.} \quad G\dot{q} \le h, \tag{4}$$

whose rows are the discrete control-barrier condition $\dot{d} \ge -\alpha\,(d - d_s)$ for every capsule pair inside the influence distance. The minimal-norm correction removes only the gap-closing component along

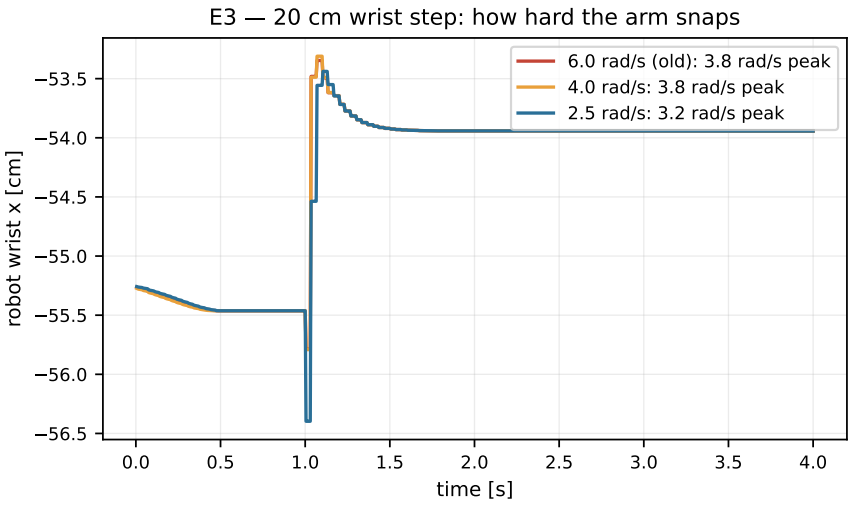


Figure 2: 20 cm wrist step under several trust rates.

each constraint gradient and leaves the tangential component untouched, so a hand driven into the torso *slides* along it rather than being pushed away. This is structurally not a potential field: there is no repulsive term, and free motion is returned bit-identical.

The distinction is testable rather than merely asserted. Driving both hands into each other and *holding* the (unreachable) target there, a potential field keeps pushing and rings, whereas a boundary layer settles. Measured: the guard parks the surface gap at exactly the safety margin, 0.0250 m, with a residual commanded joint velocity of $0.0000\,\mathrm{rad\,s^{-1}}$ (maximum 0.000). Palm separation is 0.082 m with the guard on against 0.014 m with it off.

The one honest caveat is a transient: the minimum gap during the approach reaches −0.028 m, i.e. the discrete barrier is briefly violated on a fast unreachable command before settling at the margin. It recovers, but it is not a hard no-penetration guarantee at this approach speed.

5.2 The guard contributes exactly zero jitter

Fig. 3 holds the hands still under seeded tracking noise and measures the commanded-joint standard deviation at steady state. With self-collision on and off the results are *identical to three decimal places* at every noise level, and zero-noise gives exactly 0.000 mrad:

Table 4: Steady-state command jitter on a static hold. The guard's contribution is identically zero; residual jitter is pure passthrough of hand-tracking noise at ∼1.8 mrad per mm, broadband (peak spectral power 0.02, i.e. no line).

tracking noise	self-collision ON	self-collision OFF
0 mm	0.000 mrad	0.000 mrad
1 mm	1.824 mrad	1.824 mrad
2 mm	3.646 mrad	3.646 mrad
4 mm	7.387 mrad	7.387 mrad

The operator's hypothesis is therefore falsified for the retargeting path. Two notes on where the perception may come from instead. First, the projector integrates its command from the measured

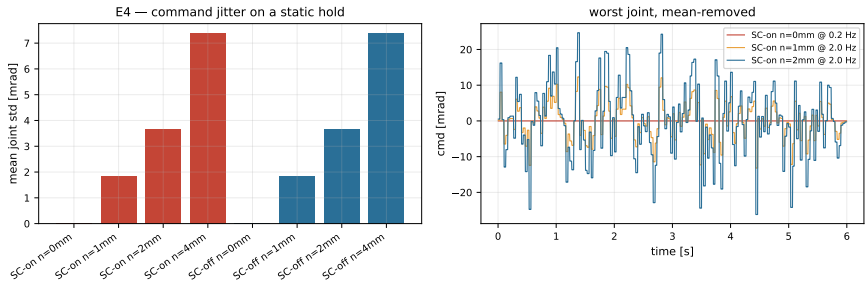


Figure 3: Command jitter on a static hold, with and without the self-collision projector, across tracking-noise levels.

pose on its first call and spends a few seconds catching up; measuring with a 2 s settle attributes ∼20 mrad of that transient to "oscillation" — for both arms of the comparison, which is what gave the artefact away. Second, a genuine oscillation mechanism does exist *on the real robot only* and outside this engine: `ros_plant.py` differentiates the arm command and low-passes it ($\alpha = 0.4$, $\tau \approx 36$ ms) before it is multiplied by the firmware's derivative gain. A lagged derivative inside a PD loop is a classical phase-lag oscillation source, and its own history records a whole-robot shake at torque-on. It is not exercised by this harness and remains open.

5.3 Capsule coverage

The guarded set is already restricted to the links that matter: forearms, hands, the ORCA wrist assemblies, torso and head, with elbow spheres. Elbow–torso pairs were removed on 2026-07-30 (J1/J2 are mechanically stop-limited and those pairs pinned the settled rest pose). Monitored pairs are hand–torso, hand–head, hand–hand, hand–forearm, forearm–forearm, and wrist-mount–torso/head.

6 Implementation and usage

- `controller/experiments/exp_vp_bimanual_latency.py` — the harness (E1 bimanual, E2 latency, E3 snap, E4 oscillation, E5 collision). Run with `--quick` or `--only E1,E4`; regenerates every figure here.
- `controller/experiments/probe_arm_crosstalk.py` — the stage bisection of Table 1.
- `controller/experiments/probe_body_yaw_swing.py` — the phantom-yaw measurement of Table 2.
- Fixes: `mabel/whole_body_retargeting/body_frame_estimator.py` (conjunction gate), `mabel/teleop_engine/hyperparams.py` (`ARM_TRUST_RATE`), and, on the client, `MabelTeleop/Tracking/HandTrackingService.swift` plus `Services/RobotController.swift`.
- Regression tests: `controller/tests/test_body_frame_estimator.py` (16) and `MabelTeleopTests/HandStalenessTests.swift` (7).

7 Limitations

- **Not verified on hardware.** Every number here is synthetic-operator and ideal-plant. The client fixes are verified only by unit test and build; neither the staleness expiry nor the hand

prediction lead has been exercised on a headset, where ARKit's actual behaviour for a hand leaving view (silence versus an untracked update) decides how much the first one matters.

- **The synthetic hands never rotate.** That is the worst case for (1) and is what exposed the defect, but it also means the 20.2° figure is an upper bound; a real operator whose wrists roll during a sweep would see less.
- **The conjunction gate has a known blind spot.** An operator who turns their torso while holding their head fixed on a target, with no wrist rotation, is suppressed. This is the same trade the module already documents for its non-rigid branch, and `Set Forward` remains the recovery.
- **Transient barrier violation.** $-0.028\,\mathrm{m}$ on a fast unreachable command (§5.1); not a hard guarantee.
- **The real-robot velocity-feedforward filter is unmeasured** and is the leading remaining oscillation candidate (§5.2).

8 References

1. A. Ames et al., "Control Barrier Functions: Theory and Applications," *European Control Conference*, 2019.
2. Y. Jiang et al., "AvatarPoser: Articulated Full-Body Pose Tracking from Sparse Motion Sensing," *ECCV*, 2022, arXiv:2207.13784.
3. C. Lu et al., "Mobile-TeleVision: Predictive Motion Priors for Humanoid Whole-Body Control," 2024, arXiv:2412.07773.
4. J. Araujo, Y. Ze, J. Xu, J. Wu, C. K. Liu, "Retargeting Matters," *ICRA*, 2026, arXiv:2510.02252.
5. G. Casiez, N. Roussel, D. Vogel, "One-Euro Filter: A Simple Speed-based Low-pass Filter for Noisy Input in Interactive Systems," *CHI*, 2012.